Equivariant CNNs vs. Standard CNNs: A Brief Tour

Group Meeting Notes

May 13, 2026

1 Review: Standard CNN

Input: grey-scale picture $\longleftrightarrow$ scalar field on a 5×5 grid.

The pipeline (repeated N times) is:

$$\underbrace{5 \times 5}_{\text{input}} \xrightarrow{\text{conv } (2 \times 2 \text{ kernel})} \underbrace{4 \times 4 \text{ scalar field}}_{\text{feature map}} \xrightarrow{\text{ReLU}} 4 \times 4 \xrightarrow{\text{pool (opt.)}} 2 \times 2$$

Example kernel: $k = \begin{pmatrix} 1 & 1 \\ 2 & -1 \end{pmatrix}$.

After N blocks: flatten to a 1×4 vector and feed it to a classifier.

2 Equivariant Neural Network (ENN)

Setup. For concreteness assume symmetry group $G = C_4$, regular representation ρ_{reg}, one layer. The group and representation can be configured *per layer*.

2.1 Convolution: the kernel is no longer a scalar field

For an ENN, the kernel at each spatial position is **not** a scalar but a 1×4 vector (in general $|G|$-dimensional, depending on the chosen group G and output representation ρ).

$$\underbrace{5 \times 5}_{\text{scalar input}} \xrightarrow{\text{conv with } k} \underbrace{4 \times 4 \text{ field}}_{\text{each pixel: } 1\times4 \text{ vector}}$$

2.2 ReLU (element-wise)

ReLU is applied independently to every channel. Example:

$$\begin{pmatrix} \binom{1}{-1} & \binom{-1}{-2} \\ \binom{2}{-1} & \binom{0}{9} \end{pmatrix} \xrightarrow{\text{ReLU}} \begin{pmatrix} \binom{1}{0} & \binom{0}{0} \\ \binom{2}{0} & \binom{0}{9} \end{pmatrix}$$

2.3 Spatial pooling (optional)

Same idea as in a standard CNN; pool the vectors element-wise (e.g. max over a 2×2 spatial window):

$$\begin{pmatrix} \binom{100}{4} & \binom{111}{3} \\ \binom{121}{2} & \binom{131}{1} \end{pmatrix} \xrightarrow{\text{spatial pool}} \binom{131}{4}$$

Stack the above $\times N$ layers; in general one may use different groups or representations per layer.

2.4 Group pooling

Group pooling acts on the *channel* dimension of each pixel (max over the $|G|$ channels), producing a scalar field that is G-invariant:

$$\begin{pmatrix} \binom{1}{2} & \binom{2}{3} \\ \binom{1}{3} & \binom{2}{4} \end{pmatrix} \xrightarrow{\text{max over channels}} \begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}$$

Then flatten $\rightarrow 1 \times 4$ vector $\rightarrow$ classifier.

Why `max`*?* A rotation of the input only permutes the $|G|$ channels of a regular field; since max is invariant under permutations, the resulting scalar features are G-invariant. This is what makes the network's predictions rotation-invariant.

3 Another Difference: Learnable Parameters

CNN: every entry of the kernel matrix k is a learnable parameter.

ENN: k is restricted by the equivariance constraint imposed by the chosen group G and representation ρ. The kernel is decomposed as

$$k \;=\; \sum_{i=1}^{d} w_i \, k_i,$$

where $\{k_1, \ldots, k_d\}$ is a basis of G-steerable kernels, pre-computed analytically and encoded in the library. *Only the coefficients w_i are learnable.*

Trade-off.

- Stricter group $\Rightarrow$ smaller basis $\Rightarrow$ more precise symmetry, fewer free parameters, less expressive.
- Looser / approximate group $\Rightarrow$ larger basis $\Rightarrow$ more powerful but less symmetric.

4 Practical Workflow

1. **Choose group and representation.** In our case $G = C_4$ or C_8, and the regular representation (chosen because it is compatible with element-wise ReLU). The library assigns the steerable kernel basis automatically.
2. **Configure architecture.** Set up input, ReLU, number of layers, and pooling — exactly the same hyperparameters as a standard CNN.
3. **Add a group-pooling layer** at the end to obtain G-invariant features.
4. **Train.** Use the output to train the classifier and the kernel coefficients via back-propagation, exactly as in a standard CNN.

5 Expected Outcome

ENNs should be more sample-efficient than CNNs on tasks whose data carries the symmetry we exploit. Concretely, on rotated handwritten digits we expect:

- Higher classification accuracy at a fixed training-set size, *or*
- Comparable accuracy with substantially less training data / time,

because rotation equivariance is built into the architecture as a hard prior rather than being learned from scratch via data augmentation.